

Phase 2: MetaInsight Engine — Single View, Single Pattern Type

Overview

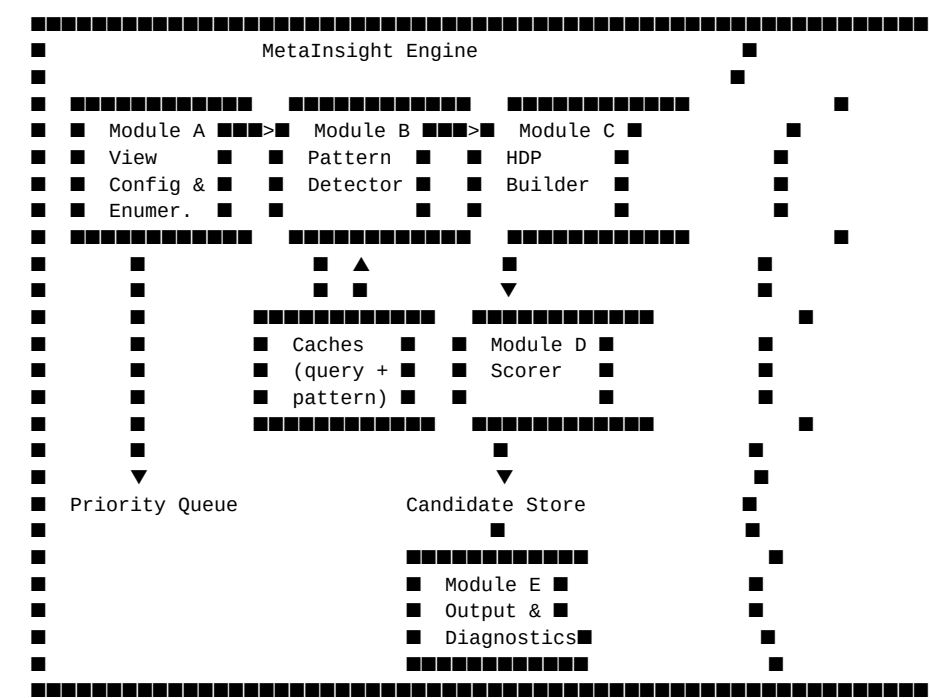
This phase implements the core MetaInsight engine end-to-end, operating on **View 1 (Claims Lifecycle)** with a single pattern type: **Outstanding #1**. The goal is to validate the full pipeline — from data scope enumeration through to scored MetaInsight candidates — before expanding to more views and pattern types in Phase 4.

Inputs: views/view1_claims_lifecycle.parquet **Outputs:**

- metainsights/view1_candidates.json — all MetaInsight candidates with scores
- reports/engine_diagnostics.txt — search space stats, cache hit rates, timing

Tech stack: Python 3.10+, pandas, numpy **Estimated runtime:** 5-15 minutes depending on hardware

Architecture



Module A: View Configuration & Data Scope Enumeration

A.1 View Configuration Schema

```
from dataclasses import dataclass, field
from typing import Optional
```

```

import pandas as pd
import numpy as np
import math
import time
import json
import heapq
from collections import Counter

@dataclass
class ViewConfig:
    """Configuration for a single analytical view."""
    name: str
    parquet_path: str

    # Column classifications
    dimensions: list[str] # categorical dimension columns
    temporal_dimensions: list[str] # temporal dimension columns
    measures: list[str] # numeric measure columns
    impact_measures: list[str] # subset of measures used for impact scoring

    # Engine parameters
    max_subspace_depth: int = 2 # max number of non-empty filters in a subspace
    tau: float = 0.5 # commonness threshold
    min_impact: float = 0.01 # prune MetaInsights with impact below this
    min_hdp_size: int = 3 # minimum HDP members to consider

```

A.2 View 1 Configuration

```

VIEW1_CONFIG = ViewConfig(
    name="Claims Lifecycle",
    parquet_path="views/view1_claims_lifecycle.parquet",

    dimensions=[
        "division", # 18 values
        "district", # 75 values
        "hospital_type", # 2 values
        "hospital_sub_type", # 8 values
        "specialty_code", # 11 values
        "disease_category", # 5 values
        "gender", # 2 values
        "age_group", # 4 values
        "admission_type", # 2 values
        "discharge_type", # 5 values
        "claim_status", # 5 values
        "preauth_status", # 4 values
        "bed_size_bucket", # 4 values
    ],

    temporal_dimensions=[
        "admission_month", # ~36 values
        "admission_quarter", # ~13 values
        "admission_year", # ~4 values
    ],

    measures=[
        "case_count",
        "amount_claimed",
        "amount_approved",
        "amount_paid",
        "length_of_stay",
        "is_emergency",
        "is_death",
        "is_lama_dama",
        "settlement_tat_days",
        "query_count",
    ]

```

```

        "base_amount",
        "computed_final_amount",
    ],

    impact_measures=[
        "case_count",          # volume
        "amount_claimed",      # financial exposure
        "amount_paid",         # actual outflow
    ],

    max_subspace_depth=2,
    tau=0.5,
    min_impact=0.01,
    min_hdp_size=3,
)

```

Column exclusions (can be added back in Phase 4):

- case_id — PK, cardinality too high
- hospital_division, hospital_district — redundant with division/district for first pass
- specialty_name, procedure_name — redundant with specialty_code
- icd_code — too granular; disease_category captures the same signal
- is_portability — 96/4 split makes HDP too unbalanced
- payment_status — 87% null
- accreditation_level — 34% null

A.3 Core Data Structures

```

@dataclass(frozen=True)
class Subspace:
    """
    A set of dimension filters.
    Empty frozenset = the full dataset (no filter).
    """
    filters: frozenset # frozenset of (str, str) tuples

    @property
    def depth(self) -> int:
        return len(self.filters)

    def __repr__(self):
        if not self.filters:
            return "{}"
        return "{" + ", ".join(f"{k}:{v}" for k, v in sorted(self.filters)) + "}"

@dataclass(frozen=True)
class DataScope:
    """A 3-tuple: (subspace, breakdown, measure)."""
    subspace: Subspace
    breakdown: str # dimension name used for group-by
    measure: str # measure name to aggregate

    def __repr__(self):
        return f"DS({self.subspace}, breakdown={self.breakdown}, measure={self.measure})"

@dataclass(frozen=True)
class Highlight:
    """
    Type-dependent encoding of a pattern's essential characteristic.

```

```

    For Outstanding #1: the breakdown value with the highest aggregate.
    """
    values: tuple # tuple for hashability

    def __repr__(self):
        return str(self.values)

@dataclass(frozen=True)
class BasicDataPattern:
    """
    A data pattern: (data_scope, type, highlight).
    type is 'NO_PATTERN' if no pattern found, 'OTHER_PATTERN' if a different
    type matched (deferred to Phase 4 multi-type evaluation).
    """
    data_scope: DataScope
    pattern_type: str
    highlight: Optional[Highlight]

```

A.4 Utility: Subspace Filtering

```

def apply_subspace(df: pd.DataFrame, subspace: Subspace) -> pd.DataFrame:
    """Filter the dataframe by the subspace's filters."""
    result = df
    for dim, val in subspace.filters:
        result = result[result[dim] == val]
    return result

```

A.5 Subspace Generation

```

def generate_subspaces(config: ViewConfig, df: pd.DataFrame) -> list[Subspace]:
    """
    Generate all subspaces from depth 0 to max_subspace_depth.

    Depth 0: {*} (no filter)
    Depth 1: {division: Lucknow}, {gender: M}, ...
    Depth 2: {division: Lucknow, gender: M}, ...
    """
    all_dims = config.dimensions # excludes temporal – temporal is for breakdown only
    subspaces = [Subspace(frozenset())] # depth 0

    # Depth 1
    depth1 = []
    for dim in all_dims:
        for val in df[dim].dropna().unique():
            depth1.append(Subspace(frozenset([(dim, val)])))
    subspaces.extend(depth1)

    # Depth 2: pairs of filters from different dimensions
    if config.max_subspace_depth >= 2:
        for i, s1 in enumerate(depth1):
            dim1 = list(s1.filters)[0][0]
            for s2 in depth1[i + 1:]:
                dim2 = list(s2.filters)[0][0]
                if dim1 != dim2:
                    combined = Subspace(s1.filters | s2.filters)
                    subspaces.append(combined)

    return subspaces

```

A.6 Data Scope Generation

```

def generate_data_scopes(subspace: Subspace, config: ViewConfig) -> list[DataScope]:
    """
    For a given subspace, generate all valid (breakdown, measure) combinations.
    Breakdown must not be a dimension already used as a filter.
    """
    filtered_dims = {dim for dim, _ in subspace.filters}
    available_breakdowns = [
        d for d in config.dimensions + config.temporal_dimensions
        if d not in filtered_dims
    ]

    scopes = []
    for breakdown in available_breakdowns:
        for measure in config.measures:
            scopes.append(DataScope(subspace, breakdown, measure))
    return scopes

```

A.7 Impact Calculator (Lazy, Cached)

```

class ImpactCalculator:
    """
    Lazily computes and caches subspace impacts.
    Impact = SUM(measure in subspace) / SUM(measure in full dataset).
    """

    def __init__(self, df: pd.DataFrame, impact_measures: list[str]):
        self._df = df
        self._totals = {m: df[m].sum() for m in impact_measures}
        self._impact_measures = impact_measures
        self._cache: dict[Subspace, dict[str, float]] = {}

    def get(self, subspace: Subspace) -> dict[str, float]:
        """Get impact values for a subspace (computing and caching if needed)."""
        if subspace in self._cache:
            return self._cache[subspace]

        filtered = apply_subspace(self._df, subspace)
        impacts = {}
        for m in self._impact_measures:
            total = self._totals[m]
            impacts[m] = filtered[m].sum() / total if total > 0 else 0.0

        self._cache[subspace] = impacts
        return impacts

    def max_impact(self, subspace: Subspace) -> float:
        """Get the maximum impact across all impact measures."""
        return max(self.get(subspace).values())

```

A.8 Priority Queue

```

def build_priority_queue(
    subspaces: list[Subspace],
    impact_calc: ImpactCalculator,
    min_impact: float,
) -> list[tuple[float, int, Subspace]]:
    """
    Build a max-heap of subspaces ordered by their max impact.
    Uses a counter as tiebreaker to avoid comparing Subspace objects
    (Subspace doesn't implement __lt__, so heapq would crash without this).
    Prunes subspaces below min_impact.
    """
    queue = []
    counter = 0

```

```

for subspace in subspaces:
    max_imp = impact_calc.max_impact(subspace)
    if max_imp >= min_impact:
        heapq.heappush(queue, (-max_imp, counter, subspace))
        counter += 1
return queue

```

Module B: Pattern Detector

B.1 Query Function

```

def query_data_scope(df: pd.DataFrame, data_scope: DataScope) -> pd.Series:
    """
    Execute: SELECT breakdown, SUM(measure) FROM df
            WHERE subspace_filters GROUP BY breakdown

    Returns a Series indexed by breakdown values.
    """
    filtered = apply_subspace(df, data_scope.subspace)
    if len(filtered) == 0:
        return pd.Series(dtype=float)

    result = filtered.groupby(data_scope.breakdown)[data_scope.measure].sum()
    return result.sort_index()

```

B.2 Query Cache

```

class QueryCache:
    """Cache for query results. Key: (subspace, breakdown, measure)."""

    def __init__(self):
        self._cache: dict[tuple, pd.Series] = {}
        self.hits = 0
        self.misses = 0

    def get(self, data_scope: DataScope) -> Optional[pd.Series]:
        key = (data_scope.subspace, data_scope.breakdown, data_scope.measure)
        if key in self._cache:
            self.hits += 1
            return self._cache[key]
        self.misses += 1
        return None

    def put(self, data_scope: DataScope, result: pd.Series):
        key = (data_scope.subspace, data_scope.breakdown, data_scope.measure)
        self._cache[key] = result

    @property
    def hit_rate(self) -> float:
        total = self.hits + self.misses
        return self.hits / total if total > 0 else 0.0

```

B.3 Pattern Cache

```

class PatternCache:
    """Cache for pattern evaluation results. Key: (data_scope, pattern_type)."""

    def __init__(self):

```

```

self._cache: dict[tuple, BasicDataPattern] = {}
self.hits = 0
self.misses = 0

def get(self, data_scope: DataScope, pattern_type: str) -> Optional[BasicDataPattern]:
    key = (data_scope, pattern_type)
    if key in self._cache:
        self.hits += 1
        return self._cache[key]
    self.misses += 1
    return None

def put(self, data_scope: DataScope, pattern_type: str, pattern: BasicDataPattern):
    key = (data_scope, pattern_type)
    self._cache[key] = pattern

@property
def hit_rate(self) -> float:
    total = self.hits + self.misses
    return self.hits / total if total > 0 else 0.0

```

B.4 Outstanding #1 Evaluator

```

def evaluate_outstanding_1(distribution: pd.Series) -> Optional[Highlight]:
    """
    Detect when one breakdown value has a significantly higher aggregate than the rest.

    Criteria:
    - At least 3 breakdown values
    - Maximum exceeds mean + 2*std of the remaining values
    - Maximum is at least 1.5x the second-highest value

    Returns Highlight with the outstanding value, or None.
    """
    if len(distribution) < 3:
        return None

    dist = distribution.dropna()
    if len(dist) < 3 or dist.sum() == 0:
        return None

    max_val = dist.max()
    max_label = dist.idxmax()

    rest = dist.drop(max_label)
    rest_mean = rest.mean()
    rest_std = rest.std()

    # Condition 1: z-score >= 2.0
    if rest_std > 0:
        z_score = (max_val - rest_mean) / rest_std
        if z_score < 2.0:
            return None
    else:
        # All others identical – max must be strictly greater
        if max_val <= rest_mean:
            return None

    # Condition 2: at least 1.5x second highest
    second_max = rest.max()
    if second_max > 0 and max_val / second_max < 1.5:
        return None

    return Highlight(values=(max_label,))

```

B.5 Pattern Detection Entry Point

```
# Pattern evaluator registry (Phase 4 adds more types here)
PATTERN_EVALUATORS = {
    "OUTSTANDING_1": evaluate_outstanding_1,
}

# Temporal-only types (not implemented in Phase 2, listed for reference)
TEMPORAL_ONLY_TYPES = {
    "TREND", "OUTLIER", "SEASONALITY", "CHANGE_POINT", "UNIMODALITY"
}

def detect_pattern(
    df: pd.DataFrame,
    data_scope: DataScope,
    pattern_type: str,
    query_cache: QueryCache,
    config: ViewConfig,
) -> BasicDataPattern:
    """
    Detect a specific pattern type in a data scope.
    Returns BasicDataPattern with the result.
    """
    # Temporal patterns only apply to temporal breakdowns
    is_temporal_breakdown = data_scope.breakdown in config.temporal_dimensions
    if pattern_type in TEMPORAL_ONLY_TYPES and not is_temporal_breakdown:
        return BasicDataPattern(data_scope, "NO_PATTERN", None)

    # Query (with cache)
    distribution = query_cache.get(data_scope)
    if distribution is None:
        distribution = query_data_scope(df, data_scope)
        query_cache.put(data_scope, distribution)

    if len(distribution) == 0:
        return BasicDataPattern(data_scope, "NO_PATTERN", None)

    # Evaluate
    evaluator = PATTERN_EVALUATORS[pattern_type]
    highlight = evaluator(distribution)

    if highlight is not None:
        return BasicDataPattern(data_scope, pattern_type, highlight)

    # Phase 2: single type, so if it fails → NO_PATTERN.
    # Phase 4: would check other types before concluding NO_PATTERN.
    return BasicDataPattern(data_scope, "NO_PATTERN", None)
```

Module C: HDP Construction & MetaInsight Identification

C.1 Extending Strategies

```
def extend_subspace(
    data_scope: DataScope,
    df: pd.DataFrame,
    config: ViewConfig,
) -> list[tuple[str, str, list[DataScope]]]:
    """
    Subspace extending: vary a dimension's filter value across all its domain values
    """
```



```

while keeping breakdown and measure fixed.

Strategy 1: For each dimension already filtered in the subspace, vary it.
Strategy 2: For subspaces below max depth, add a new dimension as a filter.

Returns list of (extending_strategy, extending_dimension, list_of_data_scopes).
"""
results = []
breakdown = data_scope.breakdown
measure = data_scope.measure
current_filters = dict(data_scope.subspace.filters)

# Strategy 1: Vary each existing filter dimension
for filter_dim in current_filters:
    other_filters = {k: v for k, v in current_filters.items() if k != filter_dim}
    other_filters_frozen = frozenset(other_filters.items())

    sibling_scopes = []
    for val in df[filter_dim].dropna().unique():
        new_filters = other_filters_frozen | frozenset([(filter_dim, val)])
        new_subspace = Subspace(new_filters)
        sibling_scopes.append(DataScope(new_subspace, breakdown, measure))

    if len(sibling_scopes) >= config.min_hdp_size:
        results.append(("subspace", filter_dim, sibling_scopes))

# Strategy 2: Add a new dimension (only if below max depth)
# For depth-0: this is the primary extending strategy – adds a dimension
# to create a sibling group. E.g., from ({*}, district, case_count),
# extend by specialty_code → 11 siblings ({specialty_code:OBG}, district, case_count), ...
# For depth-1: adds a second filter to create depth-2 siblings.
if data_scope.subspace.depth < config.max_subspace_depth:
    filtered_dims = set(current_filters.keys())
    candidate_dims = [
        d for d in config.dimensions
        if d not in filtered_dims and d != breakdown
    ]

    for extend_dim in candidate_dims:
        sibling_scopes = []
        for val in df[extend_dim].dropna().unique():
            new_filters = data_scope.subspace.filters | frozenset([(extend_dim, val)])
            new_subspace = Subspace(new_filters)
            sibling_scopes.append(DataScope(new_subspace, breakdown, measure))

        if len(sibling_scopes) >= config.min_hdp_size:
            results.append(("subspace", extend_dim, sibling_scopes))

return results

def extend_measure(
    data_scope: DataScope,
    config: ViewConfig,
) -> list[tuple[str, str, list[DataScope]]]:
    """
    Measure extending: keep subspace and breakdown, vary the measure.
    """
    sibling_scopes = []
    for m in config.measures:
        sibling_scopes.append(DataScope(data_scope.subspace, data_scope.breakdown, m))

    if len(sibling_scopes) >= config.min_hdp_size:
        return [("measure", "measure", sibling_scopes)]
    return []

```

```

def extend_breakdown(
    data_scope: DataScope,
    config: ViewConfig,
) -> list[tuple[str, str, list[DataScope]]]:
    """
    Breakdown extending: keep subspace and measure, vary breakdown across
    temporal dimensions only (per the paper's constraint).

    NOTE: With only 3 temporal dimensions (month, quarter, year), this always
    produces exactly 3 siblings – the minimum viable HDP. MetaInsights from
    this strategy will be rare, especially with Outstanding #1 where the
    highlight labels differ across temporal grains (e.g., "2024-07" vs "2024Q3"
    vs "2024"). This becomes more productive in Phase 4 with pattern types
    like Trend where highlights are grain-independent (e.g., "upward").
    """
    sibling_scopes = []
    for b in config.temporal_dimensions:
        sibling_scopes.append(DataScope(data_scope.subspace, b, data_scope.measure))

    if len(sibling_scopes) >= config.min_hdp_size:
        return [("breakdown", "temporal_grain", sibling_scopes)]
    return []

```

C.2 MetaInsight Candidate Data Structure

```

@dataclass
class MetaInsightCandidate:
    """A candidate MetaInsight ready for scoring."""
    # HDP definition
    extending_strategy: str      # "subspace", "measure", or "breakdown"
    extending_dimension: str     # which dimension was varied
    pattern_type: str           # e.g., "OUTSTANDING_1"
    breakdown: str              # shared breakdown (or "(varies)" for breakdown-extending)
    measure: str                # shared measure (or "(varies)" for measure-extending)
    base_subspace: Subspace     # the subspace part that was NOT varied

    # Partition results
    commonness_sets: list[dict] # list of {highlight, pattern_type, members, count, proportion}
    exceptions: list[dict]      # list of {member_label, category, highlight, pattern_type}
    hdp_size: int               # total members in the HDP

    # Member subspaces (for impact computation, not serialised)
    hdp_member_subspaces: list = field(default_factory=list)

    # Scores (filled by Module D)
    conciseness: float = 0.0
    impact: float = 0.0
    score: float = 0.0
    impact_measure_used: str = ""

    def to_dict(self) -> dict:
        """Serialise for JSON output (excludes hdp_member_subspaces)."""
        return {
            "extending_strategy": self.extending_strategy,
            "extending_dimension": self.extending_dimension,
            "pattern_type": self.pattern_type,
            "breakdown": self.breakdown,
            "measure": self.measure,
            "base_subspace": str(self.base_subspace),
            "hdp_size": self.hdp_size,
            "commonness_sets": self.commonness_sets,
            "exceptions": self.exceptions,
            "conciseness": self.conciseness,
            "impact": self.impact,
            "score": self.score,

```

```

        "impact_measure_used": self.impact_measure_used,
    }

```

C.3 HDP Evaluation

```

def evaluate_hdp(
    hdp_scopes: list[DataScope],
    pattern_type: str,
    extending_strategy: str,
    extending_dimension: str,
    df: pd.DataFrame,
    config: ViewConfig,
    query_cache: QueryCache,
    pattern_cache: PatternCache,
) -> Optional[MetaInsightCandidate]:
    """
    Evaluate all data scopes in an HDP, partition by similarity,
    identify commonness(es) and exceptions.
    Returns MetaInsightCandidate if valid, else None.
    """
    n = len(hdp_scopes)
    patterns = []

    for ds in hdp_scopes:
        # Check pattern cache
        cached = pattern_cache.get(ds, pattern_type)
        if cached is not None:
            patterns.append(cached)
        else:
            pattern = detect_pattern(df, ds, pattern_type, query_cache, config)
            pattern_cache.put(ds, pattern_type, pattern)
            patterns.append(pattern)

    # --- Pruning 1: early termination ---
    # After evaluating j members, check if commonness is still achievable
    j = len(patterns)
    if j > n * (1 - config.tau):
        valid_patterns = [
            (p.pattern_type, p.highlight)
            for p in patterns
            if p.pattern_type not in ("NO_PATTERN", "OTHER_PATTERN")
        ]
        remaining = n - j

        if not valid_patterns:
            # No valid patterns found yet – can remaining alone form a commonness?
            if remaining / n < config.tau:
                return None # impossible
        else:
            counts = Counter(valid_patterns)
            max_count = counts.most_common(1)[0][1]
            if (max_count + remaining) / n < config.tau:
                return None # largest group can't reach threshold

    # --- Partition by similarity: group by (type, highlight) ---
    groups: dict[tuple, list] = {}
    no_pattern_members = []
    other_pattern_members = []

    for pattern in patterns:
        if pattern.pattern_type == "NO_PATTERN":
            no_pattern_members.append(pattern)
        elif pattern.pattern_type == "OTHER_PATTERN":
            other_pattern_members.append(pattern)
        else:

```

```

        key = (pattern.pattern_type, pattern.highlight)
        if key not in groups:
            groups[key] = []
        groups[key].append(pattern)

# --- Identify commonness sets (proportion > tau) ---
commonness_sets = []
exception_patterns = []

for (ptype, highlight), members in groups.items():
    proportion = len(members) / n
    if proportion > config.tau:
        member_labels = [
            _extract_member_label(m.data_scope, extending_strategy, extending_dimension)
            for m in members
        ]
        commonness_sets.append({
            "highlight": str(highlight),
            "pattern_type": ptype,
            "members": member_labels,
            "count": len(members),
            "proportion": proportion,
        })
    else:
        exception_patterns.extend(members)

exception_patterns.extend(no_pattern_members)
exception_patterns.extend(other_pattern_members)

# Must have at least one commonness
if not commonness_sets:
    return None

# --- Categorise exceptions ---
exceptions = []
for ep in exception_patterns:
    label = _extract_member_label(ep.data_scope, extending_strategy, extending_dimension)

    if ep.pattern_type == "NO_PATTERN":
        category = "NO_PATTERN"
    elif ep.pattern_type == "OTHER_PATTERN":
        category = "TYPE_CHANGE"
    else:
        category = "HIGHLIGHT_CHANGE"

    exceptions.append({
        "member_label": label,
        "category": category,
        "highlight": str(ep.highlight) if ep.highlight else None,
        "pattern_type": ep.pattern_type,
    })

# --- Build candidate ---
ref_scope = hdp_scopes[0]

return MetaInsightCandidate(
    extending_strategy=extending_strategy,
    extending_dimension=extending_dimension,
    pattern_type=pattern_type,
    breakdown=ref_scope.breakdown if extending_strategy != "breakdown" else "(varies)",
    measure=ref_scope.measure if extending_strategy != "measure" else "(varies)",
    base_subspace=Subspace(
        frozenset(
            (k, v) for k, v in ref_scope.subspace.filters
            if k != extending_dimension
        )
    )
)

```

```

        ) if extending_strategy == "subspace" else ref_scope.subspace,
        commonness_sets=commonness_sets,
        exceptions=exceptions,
        hdp_size=n,
        hdp_member_subspaces=[ds.subspace for ds in hdp_scopes],
    )

def _extract_member_label(
    data_scope: DataScope,
    extending_strategy: str,
    extending_dimension: str,
) -> str:
    """Extract the label identifying this member within its HDP."""
    if extending_strategy == "subspace":
        for dim, val in data_scope.subspace.filters:
            if dim == extending_dimension:
                return val
        return "?"
    elif extending_strategy == "measure":
        return data_scope.measure
    elif extending_strategy == "breakdown":
        return data_scope.breakdown
    return "?"

```

Module D: Scorer

D.1 Conciseness (Paper Equations 13-16)

```

def compute_conciseness(candidate: MetaInsightCandidate, config: ViewConfig) -> float:
    """
    Entropy-based conciseness score.

    
$$S = -(\sum(\alpha_i * \log_2(\alpha_i)) + r * \sum(\beta_j * \log_2(\beta_j)))$$


    Where  $\alpha_i$  = proportion of each commonness,
            $\beta_j$  = proportion of each exception category.

    Conciseness =  $1 - (S + \gamma * I(\text{no\_exceptions})) / S_{\text{star}}$ 

    Parameters (from the paper):
        r = 1.0      (balancing parameter between commonness and exception complexity)
        k = 3        (number of exception categories: HIGHLIGHT_CHANGE, TYPE_CHANGE, NO_PATTERN)
        gamma = 0.1  (actionability regularisation – penalises all-commonness results)
    """
    n = candidate.hdp_size
    tau = config.tau
    r = 1.0
    k = 3
    gamma = 0.1

    # Commonness proportions
    alphas = [cs["proportion"] for cs in candidate.commonness_sets]

    # Exception category proportions
    exc_categories: dict[str, int] = {}
    for exc in candidate.exceptions:
        cat = exc["category"]
        exc_categories[cat] = exc_categories.get(cat, 0) + 1
    betas = [count / n for count in exc_categories.values()]

```

```

# Entropy S
s = 0.0
for a in alphas:
    if a > 0:
        s -= a * math.log2(a)
for b in betas:
    if b > 0:
        s -= r * b * math.log2(b)

# Actionability regularisation
has_exceptions = len(candidate.exceptions) > 0
indicator = 0.0 if has_exceptions else 1.0
s_reg = s + gamma * indicator

# Upper bound S* (Lemma 4.1)
# For default params (tau=0.5, k=3, r=1): threshold=e≈2.718, k>=threshold,
# so second branch applies: S* ≈ 1.792
threshold = (1 - tau) * math.e / (tau ** (1.0 / r))

if k < threshold:
    tau_r = tau ** (1.0 / r)
    s_star = (
        -math.log2(tau)
        + r * (k * tau_r / math.e) * math.log2(math.e / (k * tau_r))
    )
else:
    s_star = (
        -tau * math.log2(tau)
        - r * (1 - tau) * math.log2((1 - tau) / k)
    )

# Normalise to [0, 1]
if s_star > 0:
    conciseness = max(0.0, 1.0 - s_reg / s_star)
else:
    conciseness = 1.0

return conciseness

```

D.2 Impact (Paper Equation 17)

```

def compute_impact(
    candidate: MetaInsightCandidate,
    impact_calc: ImpactCalculator,
    config: ViewConfig,
) -> tuple[float, str]:
    """
    Compute impact per Equation 17: ImpactHDS = sum of Impact(ds) for all ds in HDS.

    For subspace-extending: member subspaces form a sibling group; sum their impacts.
    For measure/breakdown-extending: members share the same subspace; use its impact.

    Returns (impact_value, impact_measure_name) for the best impact measure.
    """
    best_impact = 0.0
    best_measure = config.impact_measures[0]

    for impact_m in config.impact_measures:
        if candidate.extending_strategy == "subspace":
            total_impact = sum(
                impact_calc.get(s).get(impact_m, 0.0)
                for s in candidate.hdp_member_subspaces
            )
            total_impact = min(1.0, total_impact) # cap at 1.0
        else:

```

```

        total_impact = impact_calc.get(candidate.base_subspace).get(impact_m, 0.0)

    if total_impact > best_impact:
        best_impact = total_impact
        best_measure = impact_m

    return best_impact, best_measure

```

D.3 Combined Score

```

def score_candidate(
    candidate: MetaInsightCandidate,
    impact_calc: ImpactCalculator,
    config: ViewConfig,
) -> MetaInsightCandidate:
    """Score = conciseness * impact. Mutates and returns the candidate."""
    candidate.conciseness = compute_conciseness(candidate, config)
    candidate.impact, candidate.impact_measure_used = compute_impact(
        candidate, impact_calc, config
    )
    candidate.score = candidate.conciseness * candidate.impact
    return candidate

```

Module E: Engine Orchestrator & Output

E.1 Main Engine Loop

```

def run_engine(
    config: ViewConfig,
    time_budget_seconds: int = 600,
) -> tuple[list[MetaInsightCandidate], dict]:
    """
    Main MetaInsight mining loop. Returns (candidates, diagnostics).

    Uses pattern-guided mining: only extends into HDPs when a pattern is found.
    Priority queue ensures high-impact subspaces are processed first.
    """
    print(f"Loading {config.parquet_path}...")
    df = pd.read_parquet(config.parquet_path)
    print(f"  {len(df):,} rows, {len(df.columns)} cols")

    # Initialise
    query_cache = QueryCache()
    pattern_cache = PatternCache()
    impact_calc = ImpactCalculator(df, config.impact_measures)
    candidates: list[MetaInsightCandidate] = []

    # Step 1: Generate subspaces
    print("Generating subspaces...")
    subspaces = generate_subspaces(config, df)
    print(f"  {len(subspaces):,} subspaces (depth 0-{config.max_subspace_depth})")

    # Step 2: Build priority queue (computes impacts lazily)
    print("Building priority queue...")
    queue = build_priority_queue(subspaces, impact_calc, config.min_impact)
    print(f"  {len(queue):,} subspaces after impact pruning")

    # Step 3: Pattern-guided mining
    start_time = time.time()
    scopes_evaluated = 0

```

```

patterns_found = 0
hdps_evaluated = 0
metainsights_found = 0

pattern_type = "OUTSTANDING_1" # Phase 2: single pattern type

print(f"\nMining (time budget: {time_budget_seconds}s)...")

while queue and (time.time() - start_time) < time_budget_seconds:
    neg_impact, _, subspace = heapq.heappop(queue)

    # Generate data scopes for this subspace
    data_scopes = generate_data_scopes(subspace, config)

    for ds in data_scopes:
        scopes_evaluated += 1

        # Detect pattern
        pattern = detect_pattern(df, ds, pattern_type, query_cache, config)
        pattern_cache.put(ds, pattern_type, pattern)

        if pattern.pattern_type == pattern_type:
            patterns_found += 1

            # Extend into HDPs
            extensions = []
            extensions.extend(extend_subspace(ds, df, config))
            extensions.extend(extend_measure(ds, config))
            extensions.extend(extend_breakdown(ds, config))

            for ext_strategy, ext_dim, hdp_scopes in extensions:
                hdps_evaluated += 1

                candidate = evaluate_hdp(
                    hdp_scopes, pattern_type,
                    ext_strategy, ext_dim,
                    df, config, query_cache, pattern_cache,
                )

                if candidate is not None:
                    score_candidate(candidate, impact_calc, config)
                    if candidate.score > 0:
                        candidates.append(candidate)
                        metainsights_found += 1

        # Progress logging
        if scopes_evaluated % 5000 == 0:
            elapsed = time.time() - start_time
            print(f" {scopes_evaluated:,} scopes | {patterns_found} patterns | "
                  f"{metainsights_found} MetaInsights | {elapsed:.1f}s")

elapsed = time.time() - start_time

# Sort by score
candidates.sort(key=lambda c: c.score, reverse=True)

# Report
print(f"\nMining complete in {elapsed:.1f}s")
print(f" Scopes evaluated:      {scopes_evaluated:,}")
print(f" Patterns found:          {patterns_found:,}")
print(f" HDPs evaluated:          {hdps_evaluated:,}")
print(f" MetaInsights found:      {metainsights_found:,}")
print(f" Query cache hit rate:    {query_cache.hit_rate:.1%}")
print(f" Pattern cache hit rate:  {pattern_cache.hit_rate:.1%}")
if candidates:
    print(f" Top score:                {candidates[0].score:.4f}")

```



```

diagnostics = {
    "elapsed": elapsed,
    "scopes_evaluated": scopes_evaluated,
    "patterns_found": patterns_found,
    "hdps_evaluated": hdps_evaluated,
    "query_cache": query_cache,
    "pattern_cache": pattern_cache,
}
return candidates, diagnostics

```

E.2 Save Candidates

```

def save_candidates(candidates: list[MetaInsightCandidate], output_path: str):
    """Save all candidates as JSON."""
    data = [c.to_dict() for c in candidates]
    with open(output_path, "w") as f:
        json.dump(data, f, indent=2, default=str)
    print(f"Saved {len(candidates)} candidates -> {output_path}")

```

E.3 Save Diagnostics

```

def save_diagnostics(
    candidates: list[MetaInsightCandidate],
    diagnostics: dict,
    output_path: str,
):
    """Save engine diagnostics report."""
    query_cache = diagnostics["query_cache"]
    pattern_cache = diagnostics["pattern_cache"]

    lines = [
        "=" * 70,
        "METAINSIGHT ENGINE DIAGNOSTICS",
        "=" * 70,
        f"Time elapsed: {diagnostics['elapsed']:.1f}s",
        f"Scopes evaluated: {diagnostics['scopes_evaluated']:,}",
        f"Patterns found: {diagnostics['patterns_found']:,}",
        f"HDPS evaluated: {diagnostics['hdps_evaluated']:,}",
        f"MetaInsights found: {len(candidates):,}",
        f"Query cache: {query_cache.hits:,} hits, {query_cache.misses:,} misses "
        f"({query_cache.hit_rate:.1%} hit rate)",
        f"Pattern cache: {pattern_cache.hits:,} hits, {pattern_cache.misses:,} misses "
        f"({pattern_cache.hit_rate:.1%} hit rate)",
        "",
        "--- Score Distribution ---",
    ]

    if candidates:
        lines.append(f"    Max:    {candidates[0].score:.4f}")
        lines.append(f"    Median: {candidates[len(candidates)//2].score:.4f}")
        lines.append(f"    Min:    {candidates[-1].score:.4f}")
    else:
        lines.append("    (no candidates)")

    lines.append("")
    lines.append("--- Top 10 MetaInsights ---")

    for i, c in enumerate(candidates[:10]):
        lines.append(f"\n    #{i+1} (score={c.score:.4f})")
        lines.append(f"        Strategy: {c.extending_strategy} on {c.extending_dimension}")
        lines.append(f"        Pattern: {c.pattern_type}")
        lines.append(f"        Breakdown: {c.breakdown}, Measure: {c.measure}")
        lines.append(f"        Base subspace: {c.base_subspace}")
        lines.append(f"        HDP size: {c.hdp_size}")

```

```

lines.append(f"    Commonness: {len(c.commonness_sets)} set(s)")
for cs in c.commonness_sets:
    lines.append(f"        - {cs['highlight']} ({cs['count']}/{c.hdp_size}, "
                  f"{cs['proportion']:.0%})")
lines.append(f"    Exceptions: {len(c.exceptions)}")
for exc in c.exceptions[:5]:
    lines.append(f"        - {exc['member_label']}: {exc['category']} "
                  f"(highlight={exc['highlight']})")
if len(c.exceptions) > 5:
    lines.append(f"        ... and {len(c.exceptions) - 5} more")
lines.append(f"    Conciseness: {c.conciseness:.4f}")
lines.append(f"    Impact: {c.impact:.4f} (via {c.impact_measure_used})")

with open(output_path, "w") as f:
    f.write("\n".join(lines))
print(f"Diagnostics saved -> {output_path}")

```

Running Phase 2

```

if __name__ == "__main__":
    import os

    os.makedirs("metainsights", exist_ok=True)
    os.makedirs("reports", exist_ok=True)

    candidates, diagnostics = run_engine(VIEW1_CONFIG, time_budget_seconds=600)

    save_candidates(candidates, "metainsights/view1_candidates.json")
    save_diagnostics(candidates, diagnostics, "reports/engine_diagnostics.txt")

```

Validation Checklist

After running the engine, verify:

- Engine completes within time budget (10 minutes)
- At least 50 MetalInsight candidates found (could be hundreds or thousands)
- Query cache hit rate > 30%
- Pattern cache hit rate > 20%
- Top-scoring MetalInsights have score > 0.1
- Top MetalInsights have both commonness AND exceptions (actionability works)
- All commonness proportions are > 0.5 (tau enforced)
- No MetalInsight has HDP size < 3 (min_hdp_size enforced)
- S_star for default params ≈ 1.792 (verify by printing once)
- Spot-check: manually verify 2-3 top MetalInsights against the raw data

What to bring back

1. metainsights/view1_candidates.json (or at least the top 20)
2. reports/engine_diagnostics.txt
3. Any errors or unexpected behaviour

Notes for Implementation

1. **Memory:** View 1 is only 22,500 rows — everything runs in-memory. 4GB+ RAM recommended as caches can grow large.
2. **Performance bottleneck:** The main cost is `apply_subspace` (DataFrame filtering) inside `query_data_scope`. The query cache mitigates this. If performance is still an issue, consider pre-computing augmented queries (paper Section 4.2.2) where a single `GROUP BY (breakdown, extending_dimension)` serves an entire HDP.
3. **Deduplication across impact measures:** A single HDP can produce multiple candidates with different scores depending on which impact measure is used. These are intentionally kept separate — Layer 3 (Phase 5) deduplicates.
4. **Phase 4 preview:** To add more pattern types:
 - Add evaluator functions to `PATTERN_EVALUATORS`
 - In `detect_pattern`, try all eligible types (temporal gate applies)
 - In HDP evaluation, patterns may have different types → `TYPE_CHANGE` exceptions
 - Add the types to `TEMPORAL_ONLY_TYPES` if they require temporal breakdown